

OSSP

Practical Week - 4

Roll No: 2520090087

Name : D. Bala Charan

Task 1:

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 1; i <= 3; i++) {
        pid = fork();

        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0) {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i, getpid(), getppid());

            sleep(i * 2);

            printf("Child %d finished.\n", i);
            exit(0);
        }
    }

    // Parent waits for children using wait()
    printf("\nParent is waiting using wait()...\n");

    for (i = 0; i < 3; i++) {
        pid = wait(NULL);
        printf("wait(): Child with PID %d has finished.\n", pid);
    }

    printf("\nAll children completed using wait().\n");

    return 0;
}
```

OUTPUT:

```
nocturnal@nocturnal:~$ nano task1.c
nocturnal@nocturnal:~$ gcc task1.c -o task1
nocturnal@nocturnal:~$ ./task1
Parent process started. PID = 549
Child 1: PID = 550, Parent PID = 549

Child 2: PID = 551, Parent PID = 549
Parent is waiting using wait()...
Child 3: PID = 552, Parent PID = 549
Child 1 finished.
wait(): Child with PID 550 has finished.
Child 2 finished.
wait(): Child with PID 551 has finished.
Child 3 finished.
wait(): Child with PID 552 has finished.

All children completed using wait().
nocturnal@nocturnal:~$
```

Waitpid()

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (pid[i] == 0) {
            printf("Child %d: PID = %d\n", i + 1, getpid());

            sleep((i + 1) * 2);

            printf("Child %d finished.\n", i + 1);
            exit(0);
        }
    }
}
```

```

// Parent waits for specific children
printf("\nParent is waiting using waitpid()...\n");

for (int i = 0; i < 3; i++) {
    waitpid(pid[i], NULL, 0);

    printf("waitpid(): Child %d with PID %d has finished.\n",
           i + 1, pid[i]);
}

printf("\nAll children completed.\n");

return 0;
}

```

OUTPUT

```

nocturnal@nocturnal:~$ nano task1_waitpid.c
nocturnal@nocturnal:~$ gcc task1_waitpid.c -o task1_waitpid
nocturnal@nocturnal:~$ ./task1_waitpid
Parent process started. PID = 723
Child 1: PID = 724
Child 2: PID = 725

Parent is waiting using waitpid()...
Child 3: PID = 726
Child 1 finished.
waitpid(): Child 1 with PID 724 has finished.
Child 2 finished.
waitpid(): Child 2 with PID 725 has finished.
Child 3 finished.
waitpid(): Child 3 with PID 726 has finished.

All children completed.
nocturnal@nocturnal:~$

```

Task-2:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent is sleeping...\n");

        sleep(30);

        printf("Parent finished.\n");
    }

    return 0;
}
```

```
nocturnal@nocturnal:~$ nano zombie.c
nocturnal@nocturnal:~$ gcc zombie.c -o zombie
nocturnal@nocturnal:~$ ./zombie
Parent process: PID = 809
Child PID = 810
Parent is sleeping...
Child process: PID = 810
Child is terminating...
```

4	S	1000	869	863	0	80	0	-	1567	do_wai	pts/0	00:00:00	bash
0	S	1000	903	869	0	80	0	-	692	hrttime	pts/0	00:00:00	zombie
1	Z	1000	904	903	0	80	0	-	0	-	pts/0	00:00:00	zombie

Without Zombie

```
GNU nano 8.7.1 zombie.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    // Child process
    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child is terminating...\n");
        exit(0);
    }

    // Parent process
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent is waiting for child...\n");
    }
}
```

```
GNU nano 8.7.1 zombie.c *
    exit(1);
}

// Child process
if (pid == 0) {
    printf("Child process: PID = %d\n", getpid());
    printf("Child is terminating...\n");
    exit(0);
}

// Parent process
else {
    printf("Parent process: PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);

    printf("Parent is waiting for child...\n");

    // Wait for the child to finish
    wait(NULL);

    printf("Child process collected. No zombie.\n");
}

return 0;
}
```

```
nocturnal@nocturnal:~$ gcc zombie.c -o zombie
nocturnal@nocturnal:~$ ./zombie
Parent process: PID = 1012
Child PID = 1013
Parent is waiting for child...
Child process: PID = 1013
Child is terminating...
Child process collected. No zombie.
nocturnal@nocturnal:~$
```

Without wait() → child becomes a zombie.

With wait() → parent collects the child → zombie is eliminated